



Rapport de Projet

LU3IN025 - Intelligence Artificielle et Jeux

Zhou Jérémy et Chen Florent

Sommaire :

- I. Introduction du sujet (p.3)
- II. Partie 1 : Problème et affectation (p.3-9)
 - Question 1 (p.4)
 - Question 2 (p.5)
 - Question 3 (p.6)
 - Question 4 (p.7)
 - Question 5 (p.9)
 - Question 6 (p.9)
- III. Partie 2 : Evolution du temps de calcul (p.10-13)
 - Question 7 (p.10)
 - Question 8 (p.11)
 - Question 9 (p.12)
 - Question 10 (p.13)
- IV. Partie 3 : Equité et PL(NE) (p.14-20)
 - Question 11 (p.14-17)
 - Question 12 (p.17-18)
 - Question 13 (p.18)
 - Question 14 (p.18)
 - Question 15 (p.19)
 - Question 16 (p.20)

I. Introduction du sujet

On s'intéresse au problème d'affectation des étudiants aux différents masters informatiques proposés par Sorbonne Université : ANDROIDE, BIM, DAC, IMA, RES, SAR, SESI, SFPN et STL.

L'objectif de ce projet est de déterminer des affectations efficaces et stables de ce problème en utilisant l'algorithme de Gale-Shapley.

Pour cela, nous avons accès aux préférences des étudiants (fichier PrefEtu.txt) ainsi qu'aux préférences des parcours (fichier PrefSpe.txt).

II. Partie 1 : Problème et affectation

Dans cette partie, on implémente les différentes fonctions nécessaires à la lecture des fichiers de préférences et on implémente les algorithmes de Gale-Shapley côté étudiant et côté parcours. De plus, on implémente aussi un algorithme permettant de détecter la présence de paire instable dans une affectation donnée.

Question 1:

```
# Partie 1

# Question 1

def read_pref_etu(f):
    fichier = open(f, "r")          # Ouverture du fichier en mode lecture "r"
    contenu = fichier.readlines()   # Liste contenant chaque ligne du fichier
    fichier.close()                 # Fermeture du fichier

    nbEtu = int(contenu[0])
    matrix = [[] for i in range(nbEtu)]
    agent=[]
    contenu = contenu[1:]           # On retire la 1ere ligne (nombre d'etudiants)

    for ligne in contenu:
        l = ligne.split()           # Liste contenant chaque element de la ligne separe par un espace
        numEtu = int(l[0])
        nomEtu = l[1]
        agent.append(l[0])
        l = l[2:]                   # On retire les 2 premiers elements (numero + nom)
        matrix[numEtu] = l

    return matrix, agent

def read_pref_spe(f):
    fichier = open(f, "r")          # Ouverture du fichier en mode lecture "r"
    contenu = fichier.readlines()   # Liste contenant chaque ligne du fichier
    fichier.close()                 # Fermeture du fichier
    agent=[]
    contenu = contenu[1:]           # On retire la 1ere ligne (nombre d'etudiants)

    # On met la capacite de chaque parcours dans une liste
    capacite = contenu[0].split()
    capacite = capacite[1:]         # On retire "Cap" de la liste
    for i in range(len(capacite)):
        capacite[i] = int(capacite[i])

    nbSpe = len(capacite)           # On recupere le nombre total de spe

    matrix = [[] for i in range(nbSpe)]

    contenu = contenu[1:]           # On retire la 1ere ligne (capacite des spe)

    for ligne in contenu:
        l = ligne.split()           # Liste contenant chaque element de la ligne separe par un espace
        numSpe = int(l[0])
        nomSpe = l[1]
        agent.append(l[0])
        l = l[2:]                   # On retire les 2 premiers elements (numero + nom)
        matrix[numSpe] = l

    return matrix, capacite, agent
```

Question 2:

Pour coder l'algorithme de Gale-Shapley, on utilise les structures suivantes :

- file
- liste
- dictionnaire

Complexité des opérations suivantes :

1. Trouver un étudiant libre à chaque itération.
 - $O(n)$ avec une structure de file en Python (défiler)
2. Étant donné un étudiant libre, trouver le prochain parcours à qui faire une proposition.
 - $O(1)$ avec une structure de liste (accès à un élément)
3. Étant donné un étudiant i et un parcours j , trouver la position de l'étudiant i dans le classement du parcours j .
 - $O(n)$ avec une structure de liste (recherche d'un élément)
avec n la taille de la liste de préférences du parcours
4. Étant donné un parcours, trouver l'étudiant le moins préféré par le parcours parmi ceux qui lui sont affectés.
 - $O(n)$ avec une structure de liste (recherche d'un élément)
avec n la taille de la liste de préférences du parcours
5. Remplacer un étudiant par un autre dans l'affectation courante d'un parcours.
 - $O(1)$ avec une structure de dictionnaire (accès à un élément)

Question 3:

Partie 1

Question 3

```
def gale_shapley_etu(maListeH,maListeF,current_man,current_women,taille_max):
    """
    maListeH -> list*list*str : les preferences des hommes(etudiant)
    maListeF -> list*list*str : les preferences des femmes(master)
    current_man -> list*str : liste des hommes
    current_women -> list*str : liste des femmes
    taille_max -> capacite maximum des preferences des femmes

    La fonction applique l algo de gale shapley du cote des etudiants

    Return : la solution de gale shapley
    """

    couple={}#la solution final de gale shapley
    taille={}#0 for j in current_women} #dictionnaire sur la capacite actuelle des femmes (master) qui est initialise a 0
    position = {}#0 for j in current_man} #dictionnaire sur la position actuelle des preferences des hommes (etudiant) qui est initialise a 0
    agent=[x for x in range(len(current_man)) ]#la file d attente des hommes(etudiant)

    #Debut de l'algo
    while agent!=[]:
        i=agent[0]
        #verifie si l'agent est libre mais theoriquement les agents sont libres
        if current_man[i] not in couple.keys():
            free_man=current_man[i]
            target=maListeH[i][position[free_man]]

            #verifie si la target n est pas encore pris ou s il reste de la place
            if (target not in couple.values()) or (taille[target]<int(taille_max[int(target)])):
                couple[free_man]=target
                taille[target]+=1
                agent.pop(0)

            #sinon il va regarder dans les preferences de la femme, pour voir qui est le meilleur pour elle
            else:
                liste_cle=[free_man]
                tmp=couple.copy()
                taille[target]=0
                position[free_man]+=1

                #cherche les autres agents qui ont le meme target que lui
                for cle,valeur in couple.items():
                    if target==valeur:
                        liste_cle.append(cle)
                        position[cle]+=1
                        tmp.pop(cle,valeur)
                        agent.append(int(cle))

                #regarde les parametres du target pour garder que ces pref
                for x in range(len(maListeF[int(target)])):
                    #regarde un par un de la gauche vers la droite, et si la preference est dans la liste de cle alors il ajoute
                    if maListeF[int(target)][x] in liste_cle:
                        tmp[maListeF[int(target)][x]]=target
                        position[maListeF[int(target)][x]]-=1
                        taille[target]+=1
                        agent.remove(int(maListeF[int(target)][x]))
                    if taille[target]>=int(taille_max[int(target)]):#quand la capacite est atteint, il arrete la boucle
                        couple=tmp.copy()
                        break

    return couple
```

La complexité de cet algorithme est : $O(n+m) + O(nm)$

avec n le nombre d'étudiants et m le nombre de master

Question 4:

```
def gale_shapley_spe(maListeH,maListeF,current_man,current_women,taille_max):
    """
    maListeH -> list*list*str : Liste des preferences des hommes(etudiant)
    maListeF -> list*list*str : Liste des preferences des femmes(master)
    current_man -> list*str : Liste des hommes (etudiant)
    current_women -> list*str : Liste des femmes (master)
    taille_max -> capacite maximum des preferences des femmes (master)

    La fonction applique l'algo de Gale-Shapley du cote des masters

    Return : L'affectation (mariage) produit par Gale-Shapley cote master
    """

    couple={j: [] for j in current_man}#la solution final de gale shapley
    position = {}: {} for j in current_man#dictionnaire sur la position actuelle des preferences des hommes (etudiant) qui est initialise a 0
    agent=[x for x in range(len(current_man))]#la file d attente des hommes(etudiant)

    # Debut de l algo
    while agent != []:
        i = agent[0] # On recupere le 1er etudiant dans la file qui n'a pas encore propose

        # Verifie si l'agent est libre (mais theoriquement les agents sont libres)
        if couple[current_man[i]] == [] or len(couple[current_man[i]]) < taille_max[i]:
            free_man = current_man[i]
            target = maListeH[i][position[free_man]]

            # Si la target est deja pris par lui meme
            if target in couple[free_man]:
                position[free_man]+=1
                continue

            # Verifie si la target n est pas encore pris
            if not(any(target in liste for liste in couple.values())):
                couple[free_man].append(maListeH[i][position[free_man]])
                # Si la capacite max n'est pas atteint, alors il augmente de 1 la position (passe au suivant)
                if len(couple[current_man[i]])<taille_max[i]:
                    position[free_man]+=1
                # Sinon il retire de la file d attente
            else:
                agent.pop(0)

            # Sinon il va regarder dans les preferences de la femme, pour voir qui est le meilleur pour elle
            else:
                liste_cle=[free_man]
                tmp=couple.copy()
                position[free_man]+=1

                # Cherche les autres agents qui ont la meme target que lui
                for cle,valeur in couple.items():
                    if target in valeur:
                        liste_cle.append(cle)
                        if (taille_max[int(cle)]-len(tmp[cle])) == 0: # Cas si il a plusieurs preferences
                            position[cle] += 1
                            tmp[cle].remove(target)
                        if int(cle) not in agent: # n ajoute pas les cles en double
                            agent.append(int(cle))
                        break

                # Regarde les parametres de target pour garder que ses pref
                for x in range(len(maListeF[int(target)])):
                    # Regarde un par un de la gauche vers la droite, et si la preference est dans la liste de cle alors il ajoute
                    if maListeF[int(target)][x] in liste_cle:
                        tmp[maListeF[int(target)][x]].append(target)
                        position[maListeF[int(target)][x]] -= 1
                    # Quand la capacite est atteint, il arrete la boucle
                    if len(couple[maListeF[int(target)][x]]) >= taille_max[int(maListeF[int(target)][x])]:
                        agent.remove(int(maListeF[int(target)][x]))
                        break
                couple = tmp.copy()

    return couple
```

La complexité de cet algorithme est : $O(n) + O(nm) = O(nm)$

avec n le nombre d'étudiants et m le nombre de master

Nous avons utilisé les structures et opérations suivantes :

	Accès à un élément	Parcours	Recherche d'un élément	Enfiler	Défiler
Liste	$O(1)$	$O(n)$	$O(n)$		
Dictionnaire	$O(1)$	$O(n)$	$O(1)$		
File	$O(1)$			$O(1)$	$O(n)$ à cause du décalage des éléments

Question 5:

Les affectations sont de la forme : {parcours : étudiant, ...} pour le côté étudiant.

Avec Gale-Shapley côté étudiant, on obtient l'affectation suivante :

{'4': '1', '0': '5', '10': '4', '1': '6', '5': '0', '3': '0', '7': '7', '9': '2', '6': '8', '2': '8', '8': '3'}

Les affectations sont de la forme : {parcours : étudiant, ...} pour le côté parcours.

Avec Gale-Shapley côté parcours, on obtient l'affectation suivante :

{'0': ['5', '3'], '1': ['4'], '2': ['9'], '3': ['8'], '4': ['10'], '5': ['1'], '6': ['0'], '7': ['7'], '8': ['6', '2']}

Question 6:

```
# Partie 1
```

```
# Question 6
```

```
def paire_instable(affectation, matrix_pref_etu, matrix_pref_spe):
    """
    affectation -> dict*list*str : Affectation (mariage parcours : etudiant)
    matrix_pref_etu -> list*list*str : Liste des preferences des hommes (master)
    matrix_pref_spe -> list*list*str : Liste des preferences des femmes (master)

    Return : Liste des paires instables dans l'affectation
    """

    liste_paires_instables = []

    # On parcourt tous les parcours et les étudiants affectés
    for parcours1, liste_etudiants in affectation.items():
        for etudiant in liste_etudiants:
            liste_pref_etu = matrix_pref_etu[int(etudiant)]

            # Récupérer l'indice du parcours actuel dans les préférences de l'étudiant
            ind_etu_pref1 = liste_pref_etu.index(parcours1)

            # Comparer parcours1 avec tous les autres parcours possibles
            for parcours2 in affectation.keys():
                if parcours2 != parcours1:
                    ind_etu_pref2 = liste_pref_etu.index(parcours2)

                    # Si l'étudiant préfère parcours2 à parcours1
                    if ind_etu_pref2 < ind_etu_pref1:
                        liste_pref_parcours = matrix_pref_spe[int(parcours2)]

                        # Vérifier si parcours2 préfère aussi cet étudiant à ses affectés
                        for etudiant_affecte in affectation[parcours2]:
                            ind_etu_parcours2 = liste_pref_parcours.index(etudiant)
                            ind_etu_affecte_parcours2 = liste_pref_parcours.index(etudiant_affecte)

                            # Si parcours2 préfère l'étudiant à un de ses actuels affectés
                            if ind_etu_parcours2 < ind_etu_affecte_parcours2:
                                if (not((parcours2, etudiant) in liste_paires_instables)):
                                    liste_paires_instables.append((parcours2, etudiant))

    return liste_paires_instables
```

III. Partie 2 : Evolution du temps de calcul

Dans cette partie, on s'intéresse à l'efficacité et à la complexité de nos programmes et on vérifie si la pratique correspond bien à nos valeurs théoriques.

Question 7:

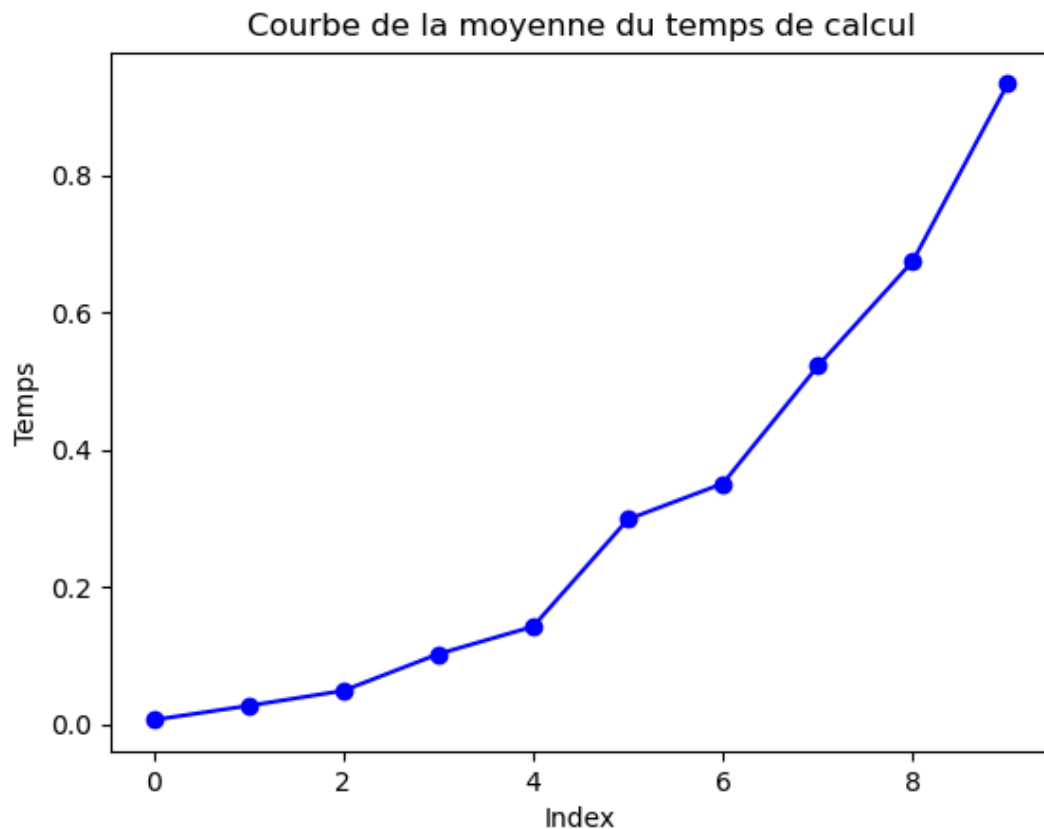
```
# Partie 2
# Question 7

def matrice_aleatoire(n_etu,n_spe):
    """
    n_etu -> nombre d etudiant
    n_spe -> nombre de spe
    Return : genere une matrice de preference -> list*list*int : random
    """
    matrice=[]
    pref=[str(x) for x in range(n_spe)]
    for i in range(n_etu):
        random.shuffle(pref) # randomiz la liste
        matrice.append(pref.copy())
    return matrice
```

Question 8:

Partie 2 : calcul des temps moyens

```
def calcul_temps():  
  
    # Configuration principale  
    n_values = range(200, 2001, 200) # Valeurs de n : de 200 à 2000 par pas de 200  
    num_tests = 10 # Nombre de tests par valeur de n  
    average_times = [] # Temps moyen pour chaque n  
    n_master=9  
    for n in n_values:  
        current_woman=[str(x) for x in range(n_master)]  
        current_man=[str(x) for x in range(n)]  
        tmp=[]  
        for _ in range(num_tests):  
            # Generation des preferences aleatoires  
            man_liste = matrice_aleatoire(n,n_master)  
            woman_liste = matrice_aleatoire(n_master,n)  
            #print(test)  
            # Generation des capacites equilibres  
            capacites = repartir_equitablement(n,n_master)  
            start_time = t.time()  
            gale_shapley_etu(man_liste, woman_liste,current_man,current_woman,capacites)  
            end_time = t.time()  
            tmp.append(end_time - start_time)  
  
            # Mesurer le temps de l'exécution de Gale-Shapley  
            average_times.append(tmp)  
    print(average_times)  
  
    # La moyenne du temps de calcul de l'algo de Gale-Shapley  
    liste_moyenne_time = ensemble_calculer_moyenne(average_times)  
    print("Liste des temps moyens : ", liste_moyenne_time)  
  
    # Trace graphique du temps moyen  
    plt.plot(liste_moyenne_time, label='Courbe des valeurs', color='b', marker='o')  
  
    # Ajout des labels et du titre  
    plt.xlabel('Index') # Axe des x (index des éléments)  
    plt.ylabel('Temps') # Axe des y (valeurs de la liste)  
    plt.title('Courbe de la moyenne du temps de calcul')  
    # Affichage du graphique  
    plt.show()  
  
    # Tracer les courbes pour chaque sous-liste dans 'data'  
    for i, subset in enumerate(average_times):  
        plt.plot(subset, label=f'Série {i+1}')  
  
    # Ajouter des labels et un titre  
    plt.xlabel('Index')  
    plt.ylabel('Temps')  
    plt.title('Courbe du temps de calcul')  
  
    # Ajouter une légende  
    plt.legend()  
  
    # Afficher le graphique  
    plt.show()
```



Question 9:

La fonction `read_pref_etu` et `read_pref_spe` ont une complexité de : $O(n)$
avec n le nombre de ligne dans le fichier

La fonction `gale_shapley_etu` a une complexité de : $O(n+m) + O(nm)$
avec n le nombre d'étudiants et m le nombre de master

La fonction `gale_shapley_spe` a une complexité de : $O(nm)$
avec n le nombre d'étudiants et m le nombre de master

La fonction `paire_instable` a une complexité de : $O(m^2 * \text{cap})$
avec m le nombre de master et cap le nombre max d'étudiants affectés au master

La fonction `matrice_aleatoire` a une complexité de : $O(n^2)$
avec n la taille de la matrice

La fonction `calcul_temps` a une complexité de : $O(n)$

Oui, les complexités correspondent bien aux complexités théoriques.

Question 10:

Théorie :

La fonction `read_pref_etu` fait 12 itérations.

(12 lignes dans le fichier)

La fonction `read_pref_spe` fait 11 itérations.

(11 lignes dans le fichier)

La fonction `gale_shapley_etu` fait $10 \cdot 8$ itérations.

(10 étudiants et 8 masters)

La fonction `gale_shapley_spe` fait $10 \cdot 8$ itérations.

(10 étudiants et 8 masters)

La fonction `paire_instable` fait $(2^2 \cdot 2) + 6^2$ itérations.

(2 masters qui peut avoir 2 étudiants et 6 masters qui ne peut avoir qu'1 étudiant)

La fonction `matrice_aleatoire` fait n itérations.

avec n le nombre d'étudiants

La fonction `calcul_temps` fait n itérations.

Pratique :

La fonction `read_pref_etu` fait 12 itérations.

La fonction `read_pref_spe` fait 11 itérations.

La fonction `gale_shapley_etu` fait 28 itérations.

La fonction `gale_shapley_spe` fait 37 itérations.

La fonction `paire_instable` fait 44 itérations.

La fonction `matrice_aleatoire` fait n itérations.

avec n le nombre d'étudiants

La fonction `calcul_temps` fait n itérations.

Oui, le nombre d'itérations correspond bien aux nombre d'itérations théoriques.

IV. Partie 3 : Équité et PL(NE)

Dans cette partie, on souhaite assurer une certaine équité entre les étudiants.

Pour cela, nous allons d'abord chercher une solution (pas forcément stable)

maximisant l'utilité minimale des étudiants grâce au score de Borda.

Puis, nous chercherons à garantir une certaine efficacité totale en plus de l'équité.

Question 11:

On cherche à savoir s'il existe une affectation où tout étudiant a un de ses k premiers choix (pour un k fixé).

On rappelle les préférences des étudiants et des parcours :

Etu0	5	7	6	8	3	2	0	1	4
Etu1	6	5	0	4	7	2	8	3	1
Etu2	4	0	7	2	8	3	1	6	5
Etu3	6	5	7	0	8	4	3	1	2
Etu4	1	6	7	5	0	2	4	8	3
Etu5	0	7	4	2	8	3	1	6	5
Etu6	5	7	6	2	8	3	0	1	4
Etu7	7	0	4	2	8	3	1	6	5
Etu8	5	7	6	2	8	3	0	1	4
Etu9	2	6	5	8	3	1	4	7	0
Etu10	6	4	0	8	3	1	5	2	7

Liste des préférences des étudiants (de + préféré à - préféré)

ANDROIDE 0	7	9	5	4	3	1	0	10	6	8	2
BIM 1	7	5	9	4	3	1	0	10	8	6	2
DAC 2	3	9	5	4	7	6	1	0	10	8	2
IMA 3	7	9	5	4	3	1	0	6	10	8	2
RES 4	10	3	0	4	5	6	7	8	9	1	2
SAR 5	1	0	3	4	5	6	7	2	9	10	8
SESI 6	0	1	3	4	5	6	7	2	8	10	9
SFPN 7	7	6	9	5	4	3	1	0	10	8	2
STL 8	1	0	3	4	5	6	7	2	9	10	8

Liste des préférences des parcours (de + préféré à - préféré)

Calculons le score de Borda des étudiants et des parcours :

	ANDROIDE 0	BIM 1	DAC 2	IMA 3	RES 4	SAR 5	SESI 6	SFPN 7	STL 8
Etu0	2	1	3	4	0	8	6	7	5
Etu1	6	0	3	1	5	7	8	4	2
Etu2	7	2	5	3	8	0	1	6	4
Etu3	5	1	0	2	3	7	8	6	4
Etu4	4	8	3	0	2	5	7	6	1
Etu5	8	2	5	3	6	0	1	7	4
Etu6	2	1	5	3	0	8	6	7	4
Etu7	7	2	5	3	6	0	1	8	4
Etu8	2	1	5	3	0	8	6	7	4
Etu9	0	3	8	4	2	6	7	1	5
Etu10	6	3	1	4	7	2	8	0	5

Score de Borda des étudiants

	Etu0	Etu1	Etu2	Etu3	Etu4	Etu5	Etu6	Etu7	Etu8	Etu9	Etu10
ANDROIDE 0	4	5	0	6	7	8	2	10	1	9	3
BIM 1	4	5	0	6	7	9	1	10	2	8	3
DAC 2	3	4	0	10	7	8	5	6	1	9	2
IMA 3	4	5	0	6	7	8	3	10	1	9	2
RES 4	8	1	0	9	7	6	5	4	3	2	10
SAR 5	9	10	3	8	7	6	5	4	0	2	1
SESI 6	10	9	3	8	7	6	5	4	2	0	1
SFPN 7	3	4	0	5	6	7	9	10	1	8	2
STL 8	9	10	3	8	7	6	5	4	0	2	1

Score de Borda des parcours

	ANDROIDE 0	BIM 1	DAC 2	IMA 3	RES 4	SAR 5	SESI 6	SFPN 7	STL 8
Etu0	6	5	6	8	8	17	16	10	14
Etu1	11	5	7	6	6	17	17	8	12
Etu2	7	2	5	3	8	3	4	6	7
Etu3	11	7	10	8	12	15	16	11	12
Etu4	11	15	10	7	9	12	14	12	8
Etu5	16	11	13	11	12	6	7	14	10
Etu6	4	2	10	6	5	13	11	16	9
Etu7	17	12	11	13	10	4	5	18	8
Etu8	3	3	6	4	3	8	8	8	4
Etu9	9	11	17	13	4	8	7	9	7
Etu10	9	6	3	6	17	3	9	2	6

Score de Borda total

Variables :

- x_{ijk} : le mariage entre l'étudiant i et le parcours j qui est dans les k premiers choix de i ($x_{ijk} = 1$ si marié, 0 sinon)
avec $i \in \{0, 1, 2, \dots, 10\}$, $j \in \{0, 1, 2, \dots, 8\}$ et k fixé.

Fonction objectif :

Pas de fonction objectif car on cherche juste si il existe une affectation où tout étudiant a un de ses k premiers choix.

Contraintes :

- Chaque étudiant i ne peut être affecté qu'à un seul parcours j .
- Chaque parcours a un nombre d'étudiants affectés maximum Cap_j .
- On ne peut affecter l'étudiant i avec le parcours j seulement si le parcours j est dans les k premiers choix de l'étudiant i .

$$\begin{aligned} \sum_{j=0}^8 x_{ijk} &= 1 && \text{pour tout } i \in \{0, 1, 2, \dots, 10\} \text{ et } k \text{ fixé} \\ \sum_{i=0}^{10} x_{ijk} &\leq Cap_j && \text{pour tout } j \in \{0, 1, 2, \dots, 8\} \text{ et } k \text{ fixé} \\ x_{ijk} &= 0 && \text{si } j \text{ n'est pas dans les } k \text{ premiers choix de } i \\ &&& i \in \{0, 1, 2, \dots, 10\}, j \in \{0, 1, 2, \dots, 8\} \text{ et } k \text{ fixé.} \end{aligned}$$

Question 12:

```
def plne():
    model = Model("Student Assignment")

    # Création du modèle
    model = LpProblem(name="Exemple2", sense=LpMinimize)

    #nombre de variables
    n=10

    #n variables x
    x = LpVariable.matrix("x", list(range(1,n+1)),lowBound=0) #n variables

    #Fonction objectif
    model += lpSum(x) #on cherche à minimiser la somme des x

    #Contraintes
    y = list(range(1,n+1))

    for i in range(n-2):
        model += (x[i]+x[i+1]+x[i+2] >= y[i])

    #Résolution du problème
    status = model.solve(solver=GLPK(msg=True,keepFiles=True)) #keepFiles permet de créer un fichier .sol contenant la solution

    # Affiche le statut de la solution (optimale, non borné, etc.)
    print("Status:", LpStatus[model.status])

    # Affiche la valeur de la fonction objectif
    print("objective=", value(model.objective))

    #Affiche les valeurs optimales des variables de décision :
    for i in range(0,n):
        print("x[" ,i,"]=",value(x[i]))
```

Non, on ne trouve pas de solution avec $k = 3$.

Question 13:

Le plus petit k tel qu'il existe une solution est $k = 5$.

Question 14:

Variables :

- x_{ij} : le mariage entre l'étudiant i et le parcours j ($x_{ij} = 1$ si marié, 0 sinon)
avec $i \in \{0, 1, 2, \dots, 10\}$ et $j \in \{0, 1, 2, \dots, 8\}$.
- SB_{ij} : le score de Borda entre l'étudiant i et le parcours j

Fonction objectif :

$$\max z = \sum_{i=0}^{10} \sum_{j=0}^8 x_{ij} * SB_{ij}$$

Contraintes :

- Chaque étudiant i ne peut être affecté qu'à un seul parcours j .
- Chaque parcours a un nombre d'étudiants affectés maximum Cap_j .

$$\begin{aligned} \sum_{j=0}^8 x_{ij} &= 1 && \text{pour tout } i \in \{0, 1, 2, \dots, 10\} \\ \sum_{i=0}^{10} x_{ij} &\leq Cap_j && \text{pour tout } j \in \{0, 1, 2, \dots, 8\} \\ i &\in \{0, 1, 2, \dots, 10\} \text{ et } j \in \{0, 1, 2, \dots, 8\}. \end{aligned}$$

D'après GLPK, on obtient :

- utilité moyenne : 6.27272727272725
- utilité minimale des étudiants : 4

Avec l'affectation: $\{(0, 3): 1, (1, 0): 1, (2, 7): 1, (3, 5): 1, (4, 2): 1, (5, 8): 1, (6, 4): 1, (7, 0): 1, (8, 1): 1, (9, 6): 1, (10, 8): 1\}$

Question 15:

Variables :

- x_{ijk} : le mariage entre l'étudiant i et le parcours j qui est dans les k premiers choix de i ($x_{ijk} = 1$ si marié, 0 sinon)
avec $i \in \{0, 1, 2, \dots, 10\}$, $j \in \{0, 1, 2, \dots, 8\}$ et $k = 5$.
- SB_{ij} : le score de Borda entre l'étudiant i et le parcours j

Fonction objectif :

$$\max z = \sum_{i=0}^{10} \sum_{j=0}^8 x_{ijk} * SB_{ij}$$

Contraintes :

- Chaque étudiant i ne peut être affecté qu'à un seul parcours j .
- Chaque parcours a un nombre d'étudiants affectés maximum Cap_j .
- On ne peut affecter l'étudiant i avec le parcours j seulement si le parcours j est dans les k premiers choix de l'étudiant i .

$$\begin{aligned} \sum_{j=0}^8 x_{ijk} &= 1 && \text{pour tout } i \in \{0, 1, 2, \dots, 10\} \text{ et } k = 5 \\ \sum_{i=0}^{10} x_{ijk} &\leq Cap_j && \text{pour tout } j \in \{0, 1, 2, \dots, 8\} \text{ et } k = 5 \\ x_{ijk} &= 0 && \text{si } j \text{ n'est pas dans les } k \text{ premiers choix de } i \\ &&& i \in \{0, 1, 2, \dots, 10\}, j \in \{0, 1, 2, \dots, 8\} \text{ et } k = 5. \end{aligned}$$

D'après GLPK, on obtient :

- l'utilité minimale des étudiants : 4
- l'utilité moyenne des étudiants : 6.0

Avec l'affectation : $\{(0, 3): 1, (1, 0): 1, (2, 7): 1, (3, 5): 1, (4, 1): 1, (5, 4): 1, (6, 8): 1, (7, 0): 1, (8, 2): 1, (9, 6): 1, (10, 8): 1\}$

Question 16:

Etudiant :

- Utilité moyenne : 1.5454545454545454
- Utilité minimale : 0
- Stable

Parcours :

- Utilité moyenne : 2.8181818181818183
- Utilité minimale : 0
- Stable

Liste des paires instables Question 13 : [('0', '6'), ('5', '0'), ('7', '0'), ('10', '0'), ('9', '2'), ('0', '5'), ('3', '5'), ('5', '4'), ('10', '4'), ('0', '7'), ('3', '7'), ('5', '7'), ('7', '7'), ('9', '8')]

Liste des paires instables Question 14 : [('0', '6'), ('5', '0'), ('7', '0'), ('10', '0'), ('9', '2'), ('0', '5'), ('3', '5'), ('5', '4'), ('10', '4'), ('0', '7'), ('3', '7'), ('5', '7'), ('7', '7'), ('9', '8')]

Liste des paires instables Question 15 : [('5', '0'), ('0', '7'), ('5', '7'), ('6', '7'), ('7', '7'), ('8', '7'), ('0', '5'), ('1', '5'), ('10', '4'), ('0', '8'), ('6', '2'), ('9', '2'), ('0', '6'), ('1', '6'), ('3', '6'), ('6', '6'), ('8', '6'), ('10', '6')]